

CSCE 606 SOFTWARE ENGINEERING · MODULE 13 · LECTURE NOTES

Maintenance and the Long Life of Code

What happens after the project ends.

MODULE OUTCOMES

Explain why software decays without being modified.

Write documentation that stays true.

Manage dependencies as an ongoing liability.

Plan for deprecation and migration.

Judge your own work by what it costs its next owner.

1 Why software decays

THE ENVIRONMENT MOVES EVEN WHEN THE CODE DOES NOT

Software does not wear out. What changes is everything around it: a dependency ships a breaking major version; an operating system removes a system call; a TLS certificate expires; a protocol version is disabled for security reasons; a third-party API alters a response format; the language runtime drops support for the version you pinned.

So a system that nobody has edited for three years may not build, may not run, and may not be deployable — and the people who understood it have moved on.

Hyrum's law compounds this: *with a sufficient number of users, every observable behaviour of your system will be depended upon by somebody* — including behaviours you never documented, never intended, and would describe as bugs. The implicit interface is everything the system does, not everything you promised.

The consequence is that 'we will not touch it' is not a maintenance strategy. Untouched code becomes unbuildable, then unrunnable, then unreplaceable, and the cost of each stage is higher than the maintenance that was avoided.

1.1 Lehman's laws

Meir Lehman studied how real systems evolve over decades. Two of his laws carry the engineering content.

Law	Statement	Implication
Continuing change	A system used in a real-world environment must be continually adapted, or it becomes progressively less satisfactory.	Maintenance is not a failure of the original design. It is the condition of being useful.
Increasing complexity	As a system evolves, its complexity increases <i>unless work is done to maintain or reduce it.</i>	This is the actionable one. Complexity growth is the default state; reversing it requires deliberate, budgeted effort.

That conditional clause reframes Module 07 entirely. Refactoring is not tidying or polish or professional pride. It is the work that offsets a documented tendency, and a project that does none is not staying still — it is losing ground.

2 Documentation

The Diátaxis framework distinguishes four kinds of documentation serving four different needs. Most projects write only one of them and are puzzled that nobody can use the system.

Kind	Serves	Question it answers	Rate of rot
Tutorial	A newcomer learning.	How do I get started?	Slow — but when it breaks, it blocks everyone new.
How-to guide	A competent user with a task.	How do I accomplish X?	Moderate.
Reference	Someone who knows what they want.	What are the parameters and return values?	Fast. Generate it from source so it cannot diverge.
Explanation	Someone trying to understand.	Why is it built this way?	Very slow , because the reasons do not change. The most valuable and the least written.

DOCUMENT WHAT CANNOT BE RECOVERED FROM THE CODE

What the code does is recoverable by reading it. Prose restating it duplicates knowledge (Module 03's DRY), so it drifts out of date and then actively misleads — and because it is documentation, it is trusted.

Why the code does it is not recoverable. Which approach was tried and abandoned, which external constraint forced the awkward structure, which production incident this guard clause commemorates, which obvious-looking simplification is wrong.

So document decisions, constraints, and rejected alternatives — architecture decision records (Module 04), commit message bodies (Module 08), and comments that explain reasoning rather than mechanism.

Wrong documentation is worse than no documentation, because a reader with none knows to read the code, and a reader with wrong documentation does not.

PREFER DOCUMENTATION THAT CANNOT LIE

Types are checked by the compiler and cannot be stale.

Tests are executable examples that fail the build if they stop being true (Module 05).

Doctests put examples in the documentation and execute them in CI, so the examples in your README are verified.

Generated reference is derived from the source and cannot diverge from it.

Assertions state invariants and check them at runtime (Module 02).

Each of these is documentation that the build system maintains for you. Reserve prose for what genuinely cannot be expressed in code — which, as above, is mostly *why*.

3 Dependencies

Cost	Detail
Security	Their vulnerabilities become yours, on their disclosure timeline, and you must respond.
Upgrade burden	Breaking changes arrive on their schedule. Falling behind makes catching up harder, and the gap compounds.
Abandonment	The maintainer loses interest or moves on. You now own a codebase you did not write and did not choose.
Transitive closure	You acquire their dependencies too, unexamined. One direct dependency can bring hundreds of indirect ones, each with all of these costs.
Supply chain risk	A compromised package executes with your application's privileges, in your build system, with your secrets in the environment.
Build time	Every dependency slows the pipeline, which Module 10 showed changes behaviour.
Licence obligations	Which someone must have actually read.

BOTH OPTIONS COST SOMETHING

Taking a dependency incurs everything above, permanently.

Writing it yourself costs implementation and ongoing maintenance, and your version will contain bugs that the mature library fixed years ago — date handling, Unicode, timezone edge cases, and character encoding are the classic examples, and reimplementing any of them is almost always a mistake.

Useful heuristics: prefer dependencies that are small in scope, that you could replace in a week if abandoned, that are widely used and actively maintained, whose source you could read if you had to, and whose licence is compatible with your use.

Be most suspicious of trivial dependencies. A package of ten lines carries every cost above and saves ten lines. The left-pad incident — where the removal of an eleven-line package broke a substantial share of the JavaScript ecosystem — demonstrated the failure mode publicly and at scale.

4 Deprecation

Removing something that people depend on is a process, not an event, and skipping steps is how systems accumulate functionality nobody dares delete.

- 1 **Announce**, with a specific removal date and a documented migration path. A warning with no date and no alternative is noise that users correctly ignore.
- 2 **Provide the replacement** before deprecating, and make migration mechanical where possible — a codemod, a shim, or a compatibility layer. Every manual step reduces the proportion of users who migrate.

- 3 **Warn at the point of use.** Deprecation warnings in logs, build output, and IDE hints reach people who never read announcements.
- 4 **Measure usage.** Instrument the deprecated path so you know who is still calling it and how often.
- 5 **Remove,** on the announced date.

Step 4 is the one that is skipped, and it is the one that makes the process work. Without usage data, nobody can demonstrate that removal is safe, so removal is deferred indefinitely, so the deprecated path is maintained forever alongside its replacement — which is strictly worse than never having deprecated it.

5 The handover test

COULD SOMEONE NEW BE PRODUCTIVE IN A WEEK?

This is the standard this course has been building toward, and it is concrete enough to check.

Can they build it from a clean machine, following only the README, without asking anyone? **Can they run the tests** and understand what failure means? **Can they make a small change** and be confident they have not broken anything? **Can they find out why** a surprising decision was made? **Can they deploy it?** **Do they know what to do** when it breaks at two in the morning?

If the answer to any of these is no, the missing knowledge is in somebody's head, and that is a single point of failure — for the project, and for the person, who can now never leave the project cleanly.

This is the test for Project 2, and it is worth applying to everything you build afterwards. It is also the most direct expression of Module 01's framing: software engineering is programming integrated over time, and the integral includes the people who come after you.

Where this course leaves you: able to judge a practice by whether it reduces the cost of the next change; able to design modules that absorb change, test them meaningfully, review code without damaging people, and ship continuously; and appropriately sceptical of confident claims in a field with more opinion than evidence — including the claims made here.

The other courses in this program are where you build things worth maintaining. **CSCE 641, 645, 647, and 649** produce codebases large enough that these practices matter. **CSCE 611 and 614** are the machine underneath. **CSCE 713** takes security seriously. And the capstone is where all of it has to hold together at the same time, which is the only real test.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Software Engineering at Google — Chapters 1, 10, 21, 22 (time, documentation, dependencies, large-scale change)	Hyrum's law, dependency management at scale, and how to make a change across millions of lines. The deprecation material is particularly good.
Diátaxis documentation framework	The four-kind taxonomy of §2. Short, practical, and it immediately explains why most project documentation fails.
Lehman — Programs, Life Cycles, and Laws of Software Evolution (free summaries)	The laws, with the empirical work behind them.
Michael Feathers — Working Effectively with Legacy Code	What to do when you inherit a system that fails the handover test.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a project of yours from more than a year ago and try to build it on a clean machine. Record every step that fails, and fix the README until it does not.
- 2 Apply the handover test to your Project 1 repository. Have someone else attempt each item and record where they get stuck.
- 3 Audit the dependencies of a project: count direct and transitive, check each direct one's last release date and open issue count, and identify any that look abandoned.
- 4 Find a trivial dependency — under fifty lines — and inline it. Assess whether the project is better off.
- 5 Write an architecture decision record for a decision in your code that would surprise a newcomer.
- 6 Classify an existing project's documentation using Diátaxis. Identify which of the four kinds is missing and write a page of it.
- 7 Find a deprecated API in a library you use. Assess the deprecation against §4's five steps and report which were done.
- 8 **Project 2 is now due.** Submit your merged open-source contribution with the full review conversation and your written reflection.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does code decay without being modified? Give three concrete causes.
- 2 State Hyrum's law and say what follows for maintaining a public interface.

- 3 Which of Lehman's laws contains the engineering content, and what clause carries it?
- 4 What should documentation record, and why is wrong documentation worse than none?
- 5 Name four forms of documentation that cannot become stale.
- 6 List four costs of taking a dependency, and say which kind to be most suspicious of.
- 7 Give the five steps of deprecation and say which is usually skipped and why that matters.
- 8 State the handover test and explain why failing it is a risk to you personally.